

Recurrent Neural Networks and Sequence Modeling

1.1 Chapter Overview and Learning Outcomes

In Chapter ??, we analyzed the artificial perceptron and multi-layer feedforward networks, while Chapter ?? introduced continuous distributed vector representations (Word2Vec and FastText). However, static feedforward architectures treat input tokens as unordered bags of words or rigid fixed-width vectors, fundamentally ignoring temporal order and sequential dependencies.

This chapter formalizes the transition from static feedforward networks to recurrent sequential architectures. We examine why standard Fully-Connected Neural Networks (FCNNs) fail on unstructured sequential text, formulate the Recurrent Neural Network (RNN) cell, derive Backpropagation Through Time (BPTT), and provide the rigorous Jacobian product proof of vanishing and exploding gradients. We inspect the empirical training dynamics of an RNN on Arthur Conan Doyle's *The Adventures of Sherlock Holmes*, trace the phase transitions across training epochs, and unpack the constant error carousel of Long Short-Term Memory (LSTM) networks [?] and Gated Recurrent Units (GRU) [?]. Finally, we analyze Sequence-to-Sequence (Seq2Seq) Encoder-Decoder pipelines [?], Teacher Forcing, and the information bottleneck that directly motivated the invention of the Transformer.

Upon completing this chapter, you will be able to:

- Identify the three fundamental architectural constraints of Fully-Connected Neural Networks on sequential unstructured data: fixed input dimensionality, lack of parameter sharing/translation invariance, and fixed output size.
- Formulate the vanilla RNN cell forward equations ($\mathbf{h}_t, \hat{\mathbf{y}}_t$) with explicit tensor dimensionality tracking across input projection $\mathbf{W}_{xh}$, recurrent transition $\mathbf{W}_{hh}$, and output emission $\mathbf{W}_{hy}$.
- Unroll recurrent architectures across discrete time steps and explain the mechanics of temporal weight sharing.
- Derive the full Jacobian product for Backpropagation Through Time (BPTT) and prove mathematically why the recurrent transition matrix spectral radius $\rho(\mathbf{W}_{hh})$ causes exponential vanishing ($\lambda < 1$) or exploding ($\lambda > 1$) gradients.
- Trace the epoch-by-epoch training dynamics of character-level RNNs, explaining the role of null-byte padding, identity passthrough, and causal lookahead blindness.
- Derive the gating equations of the LSTM cell (forget gate $\mathbf{f}_t$, input gate $\mathbf{i}_t$, candidate cell $\tilde{\mathbf{c}}_t$, additive highway $\mathbf{c}_t$, and output gate $\mathbf{o}_t$) and prove the Constant Error Carousel ($\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} = \mathbf{f}_t$).
- Contrast LSTM with the Gated Recurrent Unit (GRU), comparing parameter counts ($4\times$ vs. $3\times$ vanilla weights) and structural trade-offs.

- Explain the limitations of one-to-one sequence mappings on non-monotonic machine translation and formulate the Encoder-Decoder (Seq2Seq) architecture with Teacher Forcing.
- Analyze the persistent information and recurrence bottlenecks that motivated Attention Mechanisms [?] and Transformers [?].

1.2 Sequential Data and Constraints of Feed-Forward Neural Networks

In standard deep learning, a Fully-Connected Neural Network (FCNN) or Multi-Layer Perceptron is represented as a cascade of weight matrices interleaved with non-linear activation functions:

$$f(\mathbf{x}) = \sigma(\mathbf{W}_{\text{out}} \text{ReLU}(\mathbf{W}_2 \text{ReLU}(\mathbf{W}_1 \mathbf{x} + \mathbf{b}_1) + \mathbf{b}_2) + \mathbf{b}_{\text{out}}). \quad (1.1)$$

While FCNNs excel on tabular datasets with fixed, predefined attributes, they exhibit three critical architectural constraints when applied to natural language and temporal sequences (Slides 2–5):

1. The Input is Expected to be Fixed in Size. The input dimensionality is rigidly fixed by the number of columns in the first weight matrix $\mathbf{W}_1 \in \mathbb{R}^{d_1 \times d_{\text{in}}}$. For structured tabular data, this constraint is natural because every sample contains the same predefined attributes. However, for unstructured sequential data, fixed input sizing is deeply problematic:

- Natural language sentences come in all lengths: from 2 words (“*Run now*”) to 80 words in complex legal contracts.
- Audio recordings exhibit variable sampling frequencies and durations.
- Images come in varying resolutions and aspect ratios.

Flattening a variable-length text sequence into a rigid vector $\mathbf{x} \in \mathbb{R}^{d_{\text{in}}}$ requires setting an arbitrary maximum length T_{max} and applying zero-padding to shorter sentences while truncating longer ones. This wastes vast memory on empty pad tokens and destroys the elastic temporal nature of human language.

2. Lack of Parameter Sharing and Translation Invariance. In an FCNN, each position in the flattened input vector is associated with its own distinct set of weight parameters. Consequently, if the same linguistic pattern occurs in different portions of the input, the network must learn that pattern multiple times from scratch:

- In computer vision, a dog is a dog regardless of whether it appears on the left or the right side of an image. An FCNN must learn a separate “dog-detector” for the left pixels and the right pixels.
- In NLP, the noun phrase “*a dog*” possesses the exact same grammatical and semantic structure whether it appears at token position 1 (“*A dog barked loudly*”) or token position 10 (“*After a long walk in the park, a dog barked*”).

Because an FCNN does not share weights across sequence positions, it is wildly inefficient and requires massive datasets to observe every linguistic pattern at every conceivable spatial index.

3. The Output is Expected to be Fixed in Size. The output dimensionality is rigidly determined by the number of rows in the final projection matrix $\mathbf{W}_{\text{out}}$. In many sequential tasks (such as machine translation, text summarization, and dialogue generation), the output sequence length is unknown a priori and varies dynamically based on the input:

English (5 words): “*It is indeed sunny today*” $\implies$ Italian (4 words): “*Effettivamente oggi è soleggiato*”.

An FCNN cannot dynamically adjust its output layer to emit variable-length token sequences.

1.3 The Recurrent Neural Network Cell and Weight Sharing

To overcome the constraints of FCNNs, **Recurrent Neural Networks (RNNs)** process sequential data element by element, maintaining an internal continuous *hidden state* $\mathbf{h}_t \in \mathbb{R}^{d_h}$ that acts as a recurrent memory summarizing the sequence history observed up to time step t (Slides 6–7).

1.3.1 Recurrent Cell Architecture

At each discrete time step $t \in \{1, 2, \dots, T\}$, the recurrent cell is characterized by two inputs and two outputs:

- **Input 1:** The current sequence element $\mathbf{x}_t \in \mathbb{R}^{d_{\text{in}}}$ (e.g., a one-hot vector or continuous embedding).
- **Input 2:** The hidden state produced in the previous time step $\mathbf{h}_{t-1} \in \mathbb{R}^{d_h}$.
- **Output 1:** The model’s prediction for the current time step $\hat{\mathbf{y}}_t \in \mathbb{R}^{d_{\text{out}}}$.
- **Output 2:** The updated hidden state $\mathbf{h}_t \in \mathbb{R}^{d_h}$, which is recursively forwarded to time step $t + 1$.

1.3.2 Mathematical Formulation with Explicit Tensor Dimensions

Internally, the vanilla RNN cell comprises an input projection layer, a recurrent state transition layer, and an output projection layer (Slide 12):

$$\underbrace{\mathbf{h}_t}_{[d_h \times 1]} = \tanh \left(\underbrace{\mathbf{W}_{hh}}_{[d_h \times d_h]} \underbrace{\mathbf{h}_{t-1}}_{[d_h \times 1]} + \underbrace{\mathbf{W}_{xh}}_{[d_h \times d_{\text{in}}]} \underbrace{\mathbf{x}_t}_{[d_{\text{in}} \times 1]} + \underbrace{\mathbf{b}_h}_{[d_h \times 1]} \right), \quad (1.2)$$

$$\underbrace{\hat{\mathbf{y}}_t}_{[V \times 1]} = \text{softmax} \left(\underbrace{\mathbf{W}_{hy}}_{[V \times d_h]} \underbrace{\mathbf{h}_t}_{[d_h \times 1]} + \underbrace{\mathbf{b}_y}_{[V \times 1]} \right). \quad (1.3)$$

1.3.3 Weight Sharing Across Time

The defining feature of RNNs is **temporal weight sharing** (Slide 13). The parameter matrices $\mathbf{W}_{xh}$, $\mathbf{W}_{hh}$, and $\mathbf{W}_{hy}$ are *strictly identical* across all time steps $t \in \{1, \dots, T\}$. Because the same weights are applied regardless of time step, the model develops **translation invariance**: if the same context occurs at step $t = 2$ or step $t = 50$, the recurrent cell behaves identically. The network learns grammatical rules (such as capitalizing the first letter after a quotation mark) once, and applies that rule everywhere in the text.

1.3.4 Unrolling the Recurrent Cell Across Time

To trace how an RNN processes a sequence, we *unroll* the recursive feedback loop across discrete time steps $t = 0, 1, \dots, T$ (Slides 8–11).

Consider the character casing task:

- Input sequence: "you are very kind, mr. holmes, but i cannot do that"
- Target sequence: "You are very kind, Mr. Holmes, but I cannot do that"

Each ASCII character is represented as a 256-dimensional one-hot vector $\mathbf{x}_t \in \{0, 1\}^{256}$:

- **Step $t = 0$:** Receives input $s_0 = "$ with initial state $\mathbf{h}_{-1} = \mathbf{0}$. The cell produces state $\mathbf{h}_0$. The network realizes: *"We just encountered an opening quote ("), which signals the beginning of dialogue; the following character should be capitalized."*
- **Step $t = 1$:** Receives input $s_1 = y$ and prior state $\mathbf{h}_0$. The network emits output $\hat{y}_1 = Y$ and produces updated state $\mathbf{h}_1$. The state records: *"We just emitted a capitalized letter at the start of dialogue; following letters in this word should probably be lowercase."*
- **Step $t = 2$:** Receives input $s_2 = o$ and prior state $\mathbf{h}_1$. The network emits output $\hat{y}_2 = o$ and produces state $\mathbf{h}_2$, continuing lowercase word progression.

Figure ?? illustrates this unrolled computational structure and the backward flow of gradients.

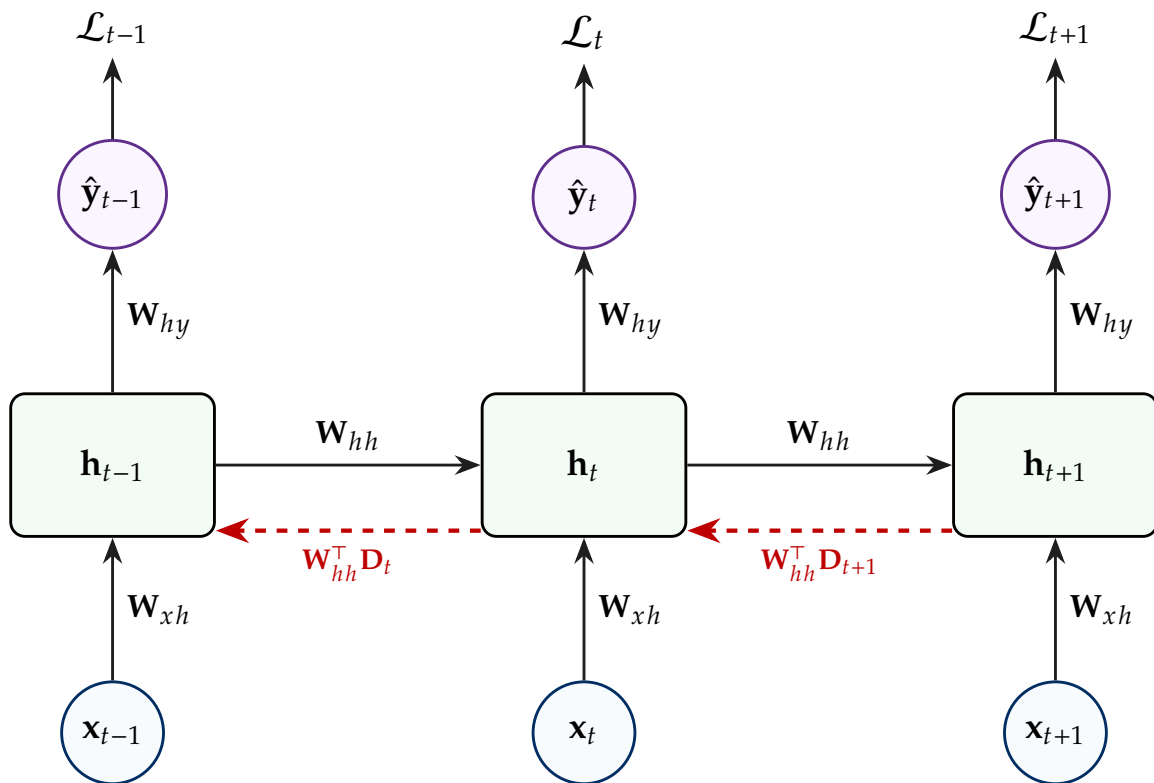


Figure 1.1: Unrolling an RNN cell across time steps $t - 1, t, t + 1$. Solid black arrows show the forward propagation of inputs $\mathbf{x}_t$ and recurrent states $\mathbf{h}_t$. Dashed red arrows trace Backpropagation Through Time (BPTT), where gradients accumulate across time via repeated multiplications by the Jacobian $\mathbf{W}_{hh}^T \mathbf{D}_t$.

1.4 Backpropagation Through Time and the Proof of Vanishing Gradients

When an RNN processes a sequence of length T , the computational graph is unrolled T times. The total sequence loss is the sum of cross-entropy losses incurred at each time step (Slides 14–17):

$$\mathcal{L} = \sum_{t=1}^T \mathcal{L}_t. \quad (1.4)$$

To optimize the shared recurrent weight matrix $\mathbf{W}_{hh}$ via Gradient Descent, we apply **Backpropagation Through Time (BPTT)**. The gradient of the total loss with respect to $\mathbf{W}_{hh}$ accumulates the contributions across all time steps:

$$\frac{\partial \mathcal{L}}{\partial \mathbf{W}_{hh}} = \sum_{t=1}^T \sum_{k=1}^t \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \frac{\partial^+ \mathbf{h}_k}{\partial \mathbf{W}_{hh}}, \quad (1.5)$$

where $\frac{\partial^+ \mathbf{h}_k}{\partial \mathbf{W}_{hh}}$ denotes the direct derivative of $\mathbf{h}_k$ with respect to $\mathbf{W}_{hh}$ treating $\mathbf{h}_{k-1}$ as constant.

The temporal transmission term $\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k}$ represents the sensitivity of state $\mathbf{h}_t$ to an earlier state $\mathbf{h}_k$ ($t > k$). By the multivariate chain rule, this sensitivity decomposes into a product of single-step Jacobian matrices:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{j=k+1}^t \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}}. \quad (1.6)$$

1.4.1 Rigorous Mathematical Proof of Gradient Pathology

We now derive the explicit single-step Jacobian and prove why vanilla RNNs suffer from vanishing and exploding gradients.

Theorem 1.1: Jacobian Product and Spectral Radius Gradient Decay

Let the recurrent state transition be $\mathbf{h}_j = \tanh(\mathbf{z}_j)$ where $\mathbf{z}_j = \mathbf{W}_{hh}\mathbf{h}_{j-1} + \mathbf{W}_{xh}\mathbf{x}_j + \mathbf{b}_h$. The single-step temporal Jacobian $\mathbf{J}_j = \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} \in \mathbb{R}^{d_h \times d_h}$ is:

$$\mathbf{J}_j = \frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} = \text{diag}(1 - \mathbf{h}_j^2) \mathbf{W}_{hh}^\top = \mathbf{D}_j \mathbf{W}_{hh}^\top, \quad (1.7)$$

where $\mathbf{D}_j = \text{diag}(1 - \tanh^2(\mathbf{z}_j))$ is a diagonal matrix containing the local derivatives of the hyperbolic tangent function. Consequently, the multi-step gradient evaluates to:

$$\frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} = \prod_{j=k+1}^t \mathbf{D}_j \mathbf{W}_{hh}^\top. \quad (1.8)$$

Let $\lambda_{\max}$ denote the spectral radius (maximum singular value) of $\mathbf{W}_{hh}^\top$. The Euclidean norm of the transmitted gradient is bounded by:

$$\left\| \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_k} \right\| \leq \left\| \frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t} \right\| (\lambda_{\max})^{t-k}. \quad (1.9)$$

Differentiating the i -th component of $\mathbf{h}_j$ with respect to the m -th component of $\mathbf{h}_{j-1}$:

$$\frac{\partial h_{j,i}}{\partial h_{j-1,m}} = \frac{\partial \tanh(z_{j,i})}{\partial z_{j,i}} \frac{\partial z_{j,i}}{\partial h_{j-1,m}} = (1 - h_{j,i}^2) W_{hh,i,m}.$$

In matrix notation, this is $\frac{\partial \mathbf{h}_j}{\partial \mathbf{h}_{j-1}} = \text{diag}(1 - \mathbf{h}_j^2) \mathbf{W}_{hh}^\top = \mathbf{D}_j \mathbf{W}_{hh}^\top$. Because $|\tanh'(z)| = 1 - \tanh^2(z) \in [0, 1]$, the operator norm of each diagonal matrix satisfies $\|\mathbf{D}_j\|_2 \leq 1$. Applying the sub-multiplicative property of matrix norms to Equation (??):

$$\begin{aligned} \left\| \frac{\partial \mathbf{h}_t}{\partial \mathbf{h}_k} \right\|_2 &= \left\| \prod_{j=k+1}^t \mathbf{D}_j \mathbf{W}_{hh}^\top \right\|_2 \leq \prod_{j=k+1}^t \|\mathbf{D}_j\|_2 \|\mathbf{W}_{hh}^\top\|_2 \\ &\leq \prod_{j=k+1}^t 1 \cdot \lambda_{\max} = (\lambda_{\max})^{t-k}. \end{aligned}$$

Multiplying by $\|\frac{\partial \mathcal{L}_t}{\partial \mathbf{h}_t}\|_2$ yields Equation (??).

Two Pathological Regimes (Slides 21–23):

1. **Vanishing Gradient** ($\lambda_{\max} < 1$): If the singular values of $\mathbf{W}_{hh}$ are slightly less than 1 (e.g., $\lambda_{\max} = 0.9$), over a temporal distance of 50 steps:

$$0.9^{50} \approx 0.00515 \quad (\text{over 99.5\% of the gradient signal is extinguished}).$$

Over 100 steps, $0.9^{100} \approx 0.000026$. The gradient decays exponentially to zero. Parameter updates from distal tokens vanish, meaning the network cannot learn long-range temporal dependencies. Saturated activations (such as $\sigma'(z) \leq 0.25$) accelerate this decay exponentially.

2. **Exploding Gradient** ($\lambda_{\max} > 1$): If the singular values exceed 1 (e.g., $\lambda_{\max} = 1.1$), over 50 steps:

$$1.1^{50} \approx 117.39.$$

Over 100 steps, $1.1^{100} \approx 13,780$. Gradients grow exponentially, destabilizing training, inducing severe numerical overflow, and producing NaN parameter corruptions. (Practitioners counter exploding gradients using heuristic *gradient clipping*: $\mathbf{g} \leftarrow \mathbf{g} \cdot \min(1, \frac{\text{threshold}}{\|\mathbf{g}\|})$).

The Long-Term Dependency Degradation (Slide 24–25). Because hidden states are rewritten at every time step without persistent memory protection, vanilla RNNs suffer from catastrophic recency bias: they remember only the most recent tokens. When tested on long-range word prediction:

*“The **mouse** was getting chased by the cat through the big house. After a long chase, the cat finally managed to catch the _____”*

the target word is unambiguously “mouse”. However, vanilla RNNs fail completely because the hidden state has overwritten the subject “mouse” across the intervening 20 tokens.

1.5 Case Correction Demonstration: Sherlock Holmes Training Dynamics

To demonstrate how an RNN learns sequential syntax in practice, Slides 18–20 document an empirical case study trained on Arthur Conan Doyle’s *The Adventures of Sherlock Holmes*:

Experimental Configuration.

- **Corpus:** Raw text from Arthur Conan Doyle's *The Adventures of Sherlock Holmes*.
- **Sequence Length:** Subsequences sliced between 10 and 100 ASCII characters.
- **Token Representation:** 256-dimensional one-hot vectors ($d_{\text{in}} = 256$).
- **Architecture:** Single-layer vanilla RNN with hidden dimension $d_h = 32$.
- **Optimization:** Adam optimizer (lr = 0.001), batch size 32, trained for 10 epochs.
- **Padding Mechanism:** Null bytes (0x00, rendered visually as X) used to pad variable-length sequences to 100 characters.

Fixed Evaluation Test Sentence. The model is evaluated after each epoch on a fixed unseen test string:

- **Input Test Sequence:** upon his pale face. "it may be so, or it may not, mr. holmes," said he, "but if you are so very sharp
- **Target Test Sequence:** upon his pale face. "It may be so, or it may not, Mr. Holmes," said he, "but if you are so very sharp

Qualitative Phase Transitions Across 10 Epochs. Table ?? records the loss trajectory and qualitative phase transitions across training:

1.6 Gated Architectures: LSTM and GRU

To solve the vanishing gradient problem and allow neural networks to retain dependencies over hundreds of time steps, researchers introduced **Gated RNNs** (Slide 26–29):

1.6.1 Long Short-Term Memory (LSTM)

Introduced by Sepp Hochreiter and Jürgen Schmidhuber [?], the **Long Short-Term Memory (LSTM)** network partitions memory into two separate states:

1. **Cell State** $\mathbf{c}_t \in \mathbb{R}^{d_h}$: The *long-term memory* highway, which runs straight down the entire chain with minimal linear interactions.
2. **Hidden State** $\mathbf{h}_t \in \mathbb{R}^{d_h}$: The *short-term memory*, which acts as the visible output emitted at each step.

Definition 1.1: *LSTM Gating Equations*

At time step t , given input $\mathbf{x}_t \in \mathbb{R}^{d_{\text{in}}}$ and previous hidden state $\mathbf{h}_{t-1} \in \mathbb{R}^{d_h}$, the LSTM

Table 1.1: Epoch-by-epoch loss progression and qualitative phase transitions on the Sherlock Holmes casing task.

Epoch	Loss	Observed Generation & Qualitative Linguistic Transition
01	3.1681	XX Phase 1 (Null-byte frequency): The model discovers that null padding bytes (0x00 / X) appear with high statistical frequency due to padding imbalance, predicting all X's.
02	2.3334	eXX Phase 2 (Vowel emergence): The model predicts e (the most frequent character in English orthography) alongside padding characters.
03	1.7436	tst X s te e eee t e e e te se ee se e e ee eee XXXee Phase 3 (Whitespace and unigrams): The model learns that whitespace characters occur regularly, predicting spaces, e, t, and s.
04	0.9519	upon his pale face. cit may be sos or it may nots mr. holresst Phase 4 (The Identity Breakthrough): A massive drop in loss occurs when the network learns to forward lowercase letters directly from input to output (w.grad captures the identity diagonal).
05-06	0.3579	upon his pale face. "it may be so, or it may not, mr. holmes," Phase 5 (Punctuation and Syntax): The model captures quotation marks ("), commas, and spaces accurately.
07-08	0.1009	upon his pale face. "It may be so, or it may not, mr. holmes," Phase 6 (Quote Capitalization): The model learns that characters immediately following opening quotes (") must be capitalized ("It).
09-10	0.0515	upon his pale face. "It may be so, or it may not, mr. Holmes," Phase 7 (Period Capitalization and Causal Blindness): The model capitalizes Holmes because it follows a period (.). Crucially, the model fails to capitalize mr. because it cannot see into the future! It does not know that m begins the title Mr. until the r. arrives.

evaluates three sigmoid gates and a candidate state:

$$\text{Forget Gate: } \mathbf{f}_t = \sigma(\mathbf{W}_f[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_f) \in (0, 1)^{d_h}, \quad (1.10)$$

$$\text{Input Gate: } \mathbf{i}_t = \sigma(\mathbf{W}_i[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_i) \in (0, 1)^{d_h}, \quad (1.11)$$

$$\text{Candidate Cell: } \tilde{\mathbf{c}}_t = \tanh(\mathbf{W}_c[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_c) \in (-1, 1)^{d_h}, \quad (1.12)$$

$$\text{Cell State Update: } \mathbf{c}_t = \mathbf{f}_t \odot \mathbf{c}_{t-1} + \mathbf{i}_t \odot \tilde{\mathbf{c}}_t \in \mathbb{R}^{d_h}, \quad (1.13)$$

$$\text{Output Gate: } \mathbf{o}_t = \sigma(\mathbf{W}_o[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_o) \in (0, 1)^{d_h}, \quad (1.14)$$

$$\text{Hidden State Update: } \mathbf{h}_t = \mathbf{o}_t \odot \tanh(\mathbf{c}_t) \in (-1, 1)^{d_h}, \quad (1.15)$$

where $\odot$ denotes the element-wise Hadamard product, and $[\mathbf{h}_{t-1}, \mathbf{x}_t] \in \mathbb{R}^{d_h+d_{in}}$ denotes vector concatenation.

Semantic Roles of the Gates (Slide 29): Each gate squashes pre-activations through a sigmoid function $\sigma \in [0, 1]$, functioning as a continuous valve:

- Multiplying by 0 $\implies$ drop/forget everything.
- Multiplying by 1 $\implies$ preserve/pass everything.
- **Forget Gate ($\mathbf{f}_t$):** Regulates what proportion of the past cell state $\mathbf{c}_{t-1}$ to discard. (e.g., when a closing quote or period arrives, the forget gate purges past sentence subjects).
- **Input Gate ($\mathbf{i}_t$):** Regulates what proportion of the new candidate information $\tilde{\mathbf{c}}_t$ to write into the long-term cell state.
- **Output Gate ($\mathbf{o}_t$):** Regulates which portions of the internal cell state $\mathbf{c}_t$ should be revealed as visible hidden state $\mathbf{h}_t$ for current output predictions.

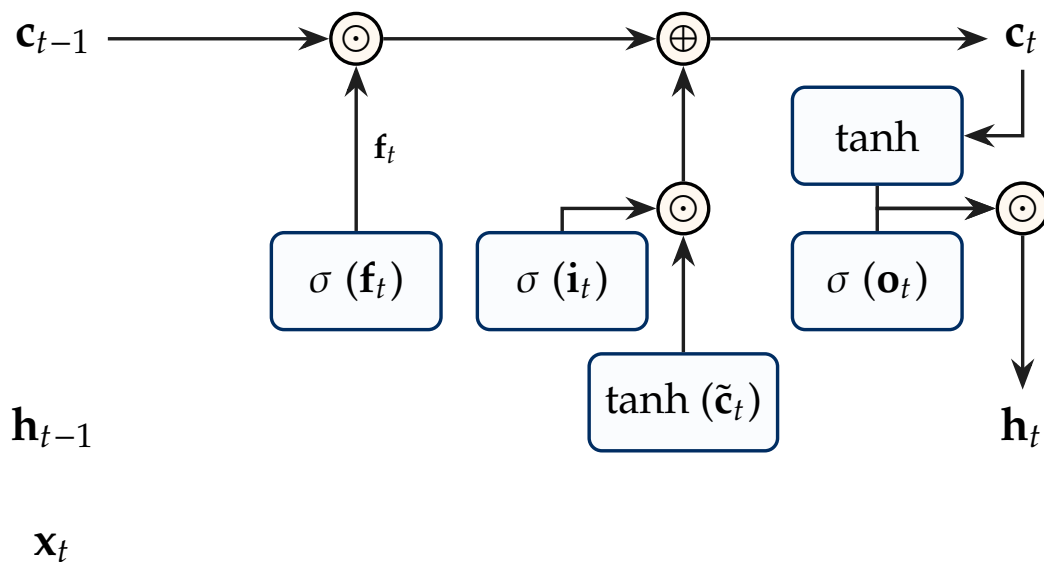


Figure 1.2: The LSTM memory cell architecture. The top horizontal path forms the uninterrupted additive cell state highway $\mathbf{c}_t$ (the Constant Error Carousel). Forget gate $\mathbf{f}_t$, input gate $\mathbf{i}_t$, and output gate $\mathbf{o}_t$ modulate information flow using multiplicative valves.

Theorem 1.2: The Constant Error Carousel Proof

In an LSTM, the gradient of the loss with respect to past cell state $\mathbf{c}_{t-1}$ contains an additive direct path:

$$\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} = \mathbf{f}_t + \frac{\partial \mathbf{f}_t}{\partial \mathbf{c}_{t-1}} \mathbf{c}_{t-1} + \frac{\partial \mathbf{i}_t}{\partial \mathbf{c}_{t-1}} \tilde{\mathbf{c}}_t + \frac{\partial \tilde{\mathbf{c}}_t}{\partial \mathbf{c}_{t-1}} \mathbf{i}_t. \quad (1.16)$$

When the network sets forget gate $\mathbf{f}_t \approx \mathbf{1}$, the leading derivative collapses to $\frac{\partial \mathbf{c}_t}{\partial \mathbf{c}_{t-1}} \approx \mathbf{I}$. Consequently, over any arbitrary temporal horizon $T - k$:

$$\frac{\partial \mathbf{c}_T}{\partial \mathbf{c}_k} = \prod_{j=k+1}^T \frac{\partial \mathbf{c}_j}{\partial \mathbf{c}_{j-1}} \approx \prod_{j=k+1}^T \mathbf{I} = \mathbf{I}. \quad (1.17)$$

Gradients flow backward across hundreds of time steps without exponential decay. This linear additive gradient highway is the **Constant Error Carousel (CEC)**.

💡 Intuition & Analogy: The Conveyor Belt of Memory

Think of the cell state $\mathbf{c}_t$ as a continuous factory conveyor belt running from the start of the sequence to the end. The standard recurrent transition in a vanilla RNN continually picks up the items, squashes them with weights and tanh, and puts a degraded version back on the belt. Over 50 time steps, the original items are unrecognizable.

In an LSTM, items on the conveyor belt are untouched by default. As the belt passes each station:

1. The forget gate $\mathbf{f}_t$ acts as a discard mechanism: if $\mathbf{f}_t \approx \mathbf{1}$, the past item rides past undisturbed; if $\mathbf{f}_t \approx \mathbf{0}$, it is swept off the belt.
2. The input gate $\mathbf{i}_t$ adds new relevant cargo $\tilde{\mathbf{c}}_t$ onto the belt via simple vector addition ($\oplus$).
3. The output gate $\mathbf{o}_t$ inspects the cargo on the belt, reads off a copy, and displays it to the outside world as hidden state $\mathbf{h}_t$, leaving the underlying cell state intact on the highway.

Because writing is additive rather than multiplicative, information can ride the conveyor belt across hundreds of words without experiencing exponential decay.

1.6.2 Gated Recurrent Unit (GRU)

Introduced by Kyunghyun Cho et al. [?], the **Gated Recurrent Unit (GRU)** simplifies the LSTM architecture by merging the cell state and hidden state into a single recurrent state $\mathbf{h}_t$ and coupling the forget and input gates into a single *update gate* $\mathbf{z}_t$:

Definition 1.2: GRU Equations

At time step t , given input $\mathbf{x}_t$ and prior state $\mathbf{h}_{t-1}$:

$$\text{Update Gate: } \mathbf{z}_t = \sigma(\mathbf{W}_z[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_z), \quad (1.18)$$

$$\text{Reset Gate: } \mathbf{r}_t = \sigma(\mathbf{W}_r[\mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_r), \quad (1.19)$$

$$\text{Candidate Hidden State: } \tilde{\mathbf{h}}_t = \tanh(\mathbf{W}_h[\mathbf{r}_t \odot \mathbf{h}_{t-1}, \mathbf{x}_t] + \mathbf{b}_h), \quad (1.20)$$

$$\text{Hidden State Interpolation: } \mathbf{h}_t = (1 - \mathbf{z}_t) \odot \mathbf{h}_{t-1} + \mathbf{z}_t \odot \tilde{\mathbf{h}}_t. \quad (1.21)$$

Exam Insights: Parameter Counting: Vanilla RNN vs. LSTM vs. GRU

A classic written examination question asks students to compare the exact parameter counts of recurrent cells for hidden dimension d and input dimension m :

- **Vanilla RNN:** Contains 1 transition set $(\mathbf{W}_{hh}, \mathbf{W}_{xh}, \mathbf{b}_h) \implies d^2 + dm + d = d(d+m+1)$.
- **GRU:** Contains 3 gate sets $(\mathbf{z}_t, \mathbf{r}_t, \tilde{\mathbf{h}}_t) \implies 3 \times d(d+m+1)$.
- **LSTM:** Contains 4 gate sets $(\mathbf{f}_t, \mathbf{i}_t, \tilde{\mathbf{c}}_t, \mathbf{o}_t) \implies 4 \times d(d+m+1)$.

The GRU reduces parameters by 25% compared to LSTM while exhibiting comparable empirical performance on moderate sequence lengths.

1.7 Sequence-to-Sequence (Seq2Seq) and Encoder-Decoder Pipelines

1.7.1 Limitations of One-to-One Sequence Mapping

Thus far, we have considered models where each input step produces exactly one output step ($|X| = |Y|$), such as character-level case correction. However, the majority of core NLP tasks (machine translation, question answering, code generation) violate this assumption (Slides 30–32):

1. **Unequal Sequence Lengths:** In machine translation, source and target languages rarely match in word count:

English (5 words): *[it] [is] [indeed] [sunny] [today]* $\implies$ Italian (4 words): *[effettivamente] [oggi] [è] [soleggiato]*.

A one-to-one recurrent model would process 5 tokens and be forced to produce 5 output tokens.

2. **Non-Monotonic Alignment and Causal Blindness:** Natural languages exhibit different word orders. In the example above, the Italian word “*effettivamente*” must be emitted at step 1, but its English counterpart “*indeed*” does not appear until step 3! In a standard one-to-one model, the cell cannot see the future, rendering early word generation impossible.

1.7.2 The Encoder-Decoder Architecture

To overcome these limitations, Sutskever et al. [?] and Cho et al. [?] introduced the **Encoder-Decoder (Seq2Seq)** architecture (Slides 33–34):

1. **Encoder:** An LSTM processes the entire variable-length source sequence $(x_1, \dots, x_N)$ step by step, updating its internal state. The final hidden state $\mathbf{h}_N$ acts as a dense, compressed summary vector encoding the semantics of the whole input sentence.

2. **Decoder:** A separate LSTM initialized with state $\mathbf{h}_N$. The decoder autoregressively emits target tokens step by step. The initial input token is the special Beginning of Sequence symbol $\langle \text{BOS} \rangle$. At each subsequent step t , the input is the token emitted at step $t - 1$. Generation continues until the model emits the special End of Sequence symbol $\langle \text{EOS} \rangle$.

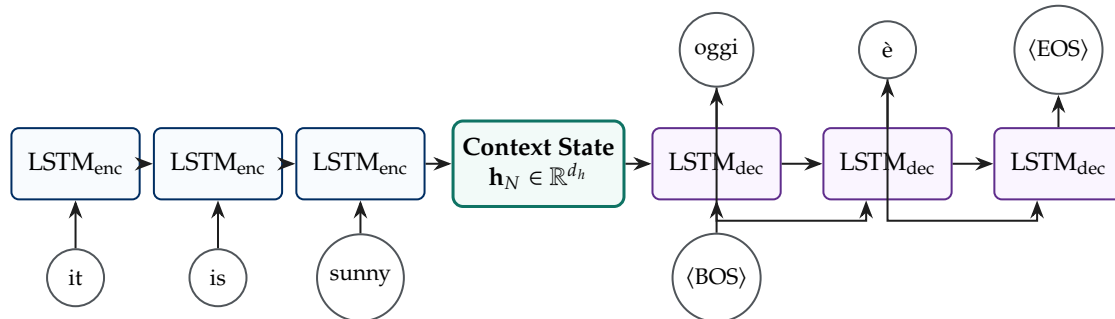


Figure 1.3: The Sequence-to-Sequence (Seq2Seq) Encoder-Decoder architecture. The Encoder LSTM compresses the variable-length source sentence into a single context state $\mathbf{h}_N$. The Decoder LSTM is initialized with $\mathbf{h}_N$ and autoregressively emits target tokens from $\langle \text{BOS} \rangle$ until emitting $\langle \text{EOS} \rangle$.

1.7.3 Teacher Forcing vs. Inference Exposure Bias

During training, the decoder's predicted token $\hat{y}_{t-1}$ is often incorrect in early epochs. If the model feeds its own faulty prediction into step t , errors compound catastrophically down the sequence.

To stabilize and accelerate training, practitioners use **Teacher Forcing** (Slide 34):

$$\mathbf{x}_{\text{dec},t} = y_{t-1}^* \quad (\text{pass true ground-truth target token instead of predicted } \hat{y}_{t-1}). \quad (1.22)$$

Exposure Bias Warning: While Teacher Forcing provides rapid convergence during training, it creates a discrepancy at inference time when ground-truth tokens are unavailable, causing models to be fragile when recovering from early generation mistakes.

1.7.4 Sherlock Holmes Case Correction Revised: One-to-One vs. Encoder-Decoder

Slides 35 and 36 evaluate the sentence:

Input: "indeed i am sherlock holmes," $\implies$ Target: "Indeed I am Sherlock Holmes,"

- **One-to-One Model:** Successfully capitalizes "Indeed" (after quote) and "Holmes" (preceded by "sherlock"). However, it completely fails on the pronoun "I" and the first name "Sherlock" because it cannot look ahead to see that "i" is a standalone word and "s" begins a proper noun.
- **Encoder-Decoder Model:** Successfully capitalizes all four targets (Indeed, I, Sherlock, Holmes) because the encoder encodes the entire sequence before generation begins.
- **Training Trade-off:** Convergence requires **1070 epochs** for Encoder-Decoder versus **10 epochs** for the one-to-one model, illustrating the optimization difficulty of learning unguided sequence-level alignment.

1.8 Persistent Bottlenecks of Gated RNNs and the Road to Transformers

While LSTMs and Encoder-Decoder architectures dominated NLP from 2014 to 2017, Slide 37 identifies four persistent bottlenecks that ultimately necessitated a new paradigm:

1. **The Information Bottleneck:** Compressing a 100-word paragraph or 200-line software function into a single fixed-size vector $\mathbf{h}_N \in \mathbb{R}^{512}$ forces catastrophic information loss. Distal dependencies cannot survive this extreme dimensional pinch point.
2. **Sequential Recurrence Bottleneck:** Because time step t strictly requires $\mathbf{h}_{t-1}$, training cannot be parallelized across the sequence dimension. GPUs are forced to execute sequentially, preventing scaling to modern web-scale pre-training datasets.
3. **Residual Gradient Degradation:** Even with gating, gradient paths across sequences longer than 100 steps experience decay and noisy gradient accumulation.

These fundamental limitations directly motivated Dzmitry Bahdanau et al. [?] to introduce the **Attention Mechanism**, enabling decoders to peer dynamically across all encoder states, and inspired Ashish Vaswani et al. [?] to eliminate recurrence entirely in “*Attention Is All You Need*”, inaugurating the Transformer era explored in Chapter 5.

External Deep Dive: Historical Milestone: From Schmidhuber’s CEC to Sutskever’s Seq2Seq

The lineage of sequence modeling represents one of the most intellectually cohesive arcs in artificial intelligence:

- In 1997, Sepp Hochreiter and Jürgen Schmidhuber published “*Long Short-Term Memory*” [?], solving the vanishing gradient problem via the Constant Error Carousel and proving that additive highways preserve error signals indefinitely.
- In 2014, Kyunghyun Cho et al. [?] introduced the Gated Recurrent Unit (GRU) and formulated RNN encoder-decoder architectures for statistical machine translation.
- Simultaneously in 2014, Ilya Sutskever, Oriol Vinyals, and Quoc V. Le [?] published “*Sequence to Sequence Learning with Neural Networks*”, demonstrating that multi-layer deep LSTMs could map sentences across languages end-to-end. Crucially, Sutskever discovered that reversing the source sentence (“ $A B C \rightarrow C B A$ ”) dramatically improved performance by shortening the initial temporal gradient distance between source and target tokens.